

Análisis comparativo profundo entre Monolito Modular y Microservicios

Aplicación al dominio de un Sistema de Gestión Académica (SGA) educativo

Estudiante: Juliana Romina Emanuel Pino
Carrera: 7mo Software "A"
Materia: Aplicaciones Distribuidas
Docente: Ing. Gleiston Cicerón Guerrero Ulloa
Fecha: 13 de junio de 2026

Resumen

El presente trabajo realiza un análisis comparativo profundo entre dos estilos de arquitectura de software ampliamente utilizados en el desarrollo de sistemas distribuidos: el *Monolito Modular* y los *Microservicios*. Se examinan sus fundamentos teóricos, principios de diseño, ventajas, limitaciones, así como criterios de decisión para su adopción. Posteriormente, se contextualiza dicho análisis dentro del dominio de un Sistema de Gestión Académica (SGA) educativo, correspondiente al proyecto integrador del equipo, proponiendo una arquitectura híbrida fundamentada en los hallazgos obtenidos. El estudio concluye que la elección entre ambos estilos no es absoluta, sino que depende del tamaño del equipo, la madurez organizacional, los requisitos de escalabilidad y la complejidad del dominio.

Palabras clave: arquitectura de software, monolito modular, microservicios, sistemas distribuidos, escalabilidad, sistema de gestión académica.

1 Introducción

El diseño de la arquitectura de software constituye una de las decisiones técnicas más determinantes en el ciclo de vida de un sistema, ya que condiciona su capacidad de evolución, mantenimiento, escalabilidad y costo operativo a largo plazo. En el contexto actual del desarrollo de software, donde los sistemas distribuidos predominan en aplicaciones empresariales y educativas, la disyuntiva entre adoptar un estilo monolítico modular o descomponer el sistema en microservicios independientes representa un dilema recurrente para los equipos de desarrollo.

Durante años, la industria promovió la migración masiva hacia microservicios como una solución universal a los problemas de escalabilidad y mantenibilidad de los sistemas monolíticos tradicionales. Sin embargo, la experiencia acumulada en la última década ha demostrado que dicha migración no siempre resulta beneficiosa, especialmente para equipos pequeños o sistemas con dominios poco definidos, donde la complejidad operacional introducida por los microservicios puede superar los beneficios obtenidos.

El presente informe tiene como objetivo realizar un análisis crítico y comparativo entre el estilo de *Monolito Modular* y el de *Microservicios*, evaluando sus características arquitectónicas, ventajas, desventajas y casos de aplicación idóneos. Posteriormente, se vincula este análisis con el proyecto integrador del equipo: un Sistema de Gestión Académica (SGA) educativo, proponiendo recomendaciones arquitectónicas fundamentadas para su desarrollo.

La estructura del documento se organiza de la siguiente manera: la Sección 2 expone el marco teórico de ambos estilos arquitectónicos; la Sección 3 desarrolla el análisis comparativo a partir de criterios técnicos definidos; la Sección 4 presenta la aplicación de los conceptos investigados al proyecto integrador (SGA); la Sección 5 expone las conclusiones; y finalmente se presentan las referencias bibliográficas consultadas.

2 Marco teórico

2.1 Monolito Modular

Un *Monolito Modular* es un estilo arquitectónico en el cual la aplicación se despliega como una única unidad ejecutable, pero internamente se organiza en módulos cohesivos y débilmente acoplados, cada uno responsable de un dominio o capacidad de negocio específica [1]. A diferencia del monolito tradicional “de bola de lodo” (*big ball of mud*), donde las responsabilidades se entremezclan sin límites claros, el monolito modular impone fronteras internas explícitas mediante módulos, paquetes o componentes bien definidos, comunicándose generalmente a través de interfaces internas (llamadas a métodos o eventos en memoria) en lugar de la red.

Entre sus características principales se destacan:

- **Despliegue unificado:** toda la aplicación se compila, prueba y despliega como un único artefacto.
- **Modularidad interna:** el código se organiza en módulos con responsabilidades acotadas, siguiendo principios como la separación de responsabilidades (*Separation of Concerns*) y el acoplamiento bajo-cohesión alta.
- **Base de datos compartida o particionada lógicamente:** los módulos pueden compartir un mismo esquema de base de datos, aunque con esquemas o tablas separadas por dominio.
- **Comunicación interna síncrona:** las interacciones entre módulos ocurren mediante llamadas directas dentro del mismo proceso, evitando la latencia de red.

2.2 Microservicios

La arquitectura de *Microservicios* consiste en estructurar una aplicación como un conjunto de servicios pequeños, autónomos y desplegables de forma independiente, cada uno organizado alrededor de una capacidad de negocio concreta y comunicándose mediante mecanismos ligeros, típicamente APIs HTTP/REST o mensajería asíncrona [2]. Cada microservicio posee su propia base de datos, su propio ciclo de despliegue y, en muchos casos, puede estar implementado en tecnologías distintas.

Entre sus características principales se destacan:

- **Despliegue independiente:** cada servicio puede actualizarse, escalarse y desplegarse sin afectar a los demás.
- **Descentralización de datos:** cada microservicio gestiona su propia base de datos (patrón *Database per Service*), evitando el acoplamiento a nivel de esquema.

- **Comunicación distribuida:** las interacciones ocurren a través de la red, lo cual introduce latencia, posibles fallos parciales y la necesidad de mecanismos de resiliencia (*circuit breakers, retries, timeouts*).
- **Escalabilidad granular:** es posible escalar horizontalmente únicamente los servicios que experimentan mayor carga, optimizando el uso de recursos.
- **Autonomía tecnológica y de equipos:** distintos equipos pueden desarrollar, desplegar y mantener servicios de forma independiente, alineándose con la Ley de Conway [3].

2.3 Relación entre ambos estilos

Resulta importante señalar que el monolito modular y los microservicios no representan extremos irreconciliables, sino puntos dentro de un espectro de descomposición arquitectónica. De hecho, una práctica recomendada en la literatura es comenzar el desarrollo de un sistema mediante un monolito modular bien diseñado, y migrar progresivamente hacia microservicios únicamente cuando existan evidencias concretas de la necesidad de escalado independiente, equipos autónomos o requisitos de despliegue diferenciados [3]. Esta estrategia reduce el riesgo de incurrir en una complejidad distribuida prematura, fenómeno descrito frecuentemente como “monolito distribuido”, donde un sistema adopta la complejidad operacional de los microservicios sin obtener sus beneficios reales.

3 Análisis comparativo

En esta sección se desarrolla el análisis crítico entre ambos estilos arquitectónicos a partir de criterios técnicos relevantes: escalabilidad, mantenibilidad, complejidad operacional, costos de infraestructura y curva de adopción para equipos.

3.1 Escalabilidad

El monolito modular permite escalar verticalmente (aumentando recursos de la instancia) o, en su forma más simple, escalar horizontalmente replicando toda la aplicación, lo cual implica que incluso los módulos con baja demanda consumen recursos adicionales en cada réplica. En contraste, los microservicios permiten una escalabilidad granular: únicamente los servicios sometidos a mayor carga –por ejemplo, un servicio de notificaciones en periodos de matrícula– pueden replicarse de forma independiente, optimizando el uso de infraestructura [4].

Sin embargo, esta ventaja de los microservicios solo se materializa cuando existe una demanda real diferenciada entre módulos. En sistemas con carga moderada y uniforme, la sobrecarga de orquestar múltiples servicios puede no justificarse frente a la simplicidad de escalar un monolito modular.

3.2 Mantenibilidad y evolución del código

El monolito modular, al mantener el código en un único repositorio y proceso, facilita la refactorización transversal, la depuración mediante trazas de pila completas y la comprensión global del sistema, especialmente para equipos pequeños. No obstante, si los límites entre módulos no

se respetan disciplinadamente, el sistema puede degenerar hacia un acoplamiento excesivo difícil de revertir.

Los microservicios, al imponer fronteras físicas (procesos y bases de datos separadas), fuerzan un acoplamiento bajo de manera más estricta, lo cual favorece la mantenibilidad a largo plazo en sistemas grandes con múltiples equipos. Sin embargo, introducen complejidad adicional: la depuración requiere trazabilidad distribuida (*distributed tracing*), las pruebas de integración se vuelven más costosas, y los cambios que atraviesan varios servicios (*cross-cutting changes*) requieren coordinación entre equipos [1].

3.3 Complejidad operacional

Este es quizás el criterio donde la diferencia resulta más marcada. El monolito modular requiere una infraestructura de despliegue relativamente simple: un servidor de aplicaciones, una base de datos y, eventualmente, un balanceador de carga para múltiples réplicas idénticas.

Los microservicios, por su parte, requieren infraestructura adicional considerable: orquestadores de contenedores (como Kubernetes), *service discovery*, *API gateways*, sistemas de monitoreo distribuido, gestión centralizada de configuración y mecanismos de resiliencia ante fallos parciales. Esta complejidad operacional representa una barrera significativa para equipos pequeños o proyectos académicos, donde el costo de mantener dicha infraestructura puede superar los beneficios obtenidos [5].

3.4 Costos de infraestructura y operación

Directamente relacionado con el punto anterior, los costos de un monolito modular tienden a ser menores en etapas iniciales, ya que no requiere múltiples entornos de ejecución, bases de datos independientes ni herramientas de observabilidad distribuida. Los microservicios, si bien pueden optimizar costos a gran escala mediante el escalado selectivo, generalmente implican mayores costos fijos de operación: licencias o instancias de orquestación, redundancia de bases de datos por servicio, y personal especializado en operaciones (DevOps/SRE).

3.5 Curva de adopción para equipos

Para equipos pequeños, con experiencia limitada en sistemas distribuidos –como es habitual en proyectos académicos integradores–, el monolito modular presenta una curva de adopción considerablemente menor. Permite al equipo enfocarse en el diseño del dominio y la calidad del código, sin distraer esfuerzos en infraestructura distribuida.

Los microservicios exigen que el equipo domine conceptos adicionales: comunicación entre servicios, consistencia eventual, transacciones distribuidas (patrón *Saga*), versionado de APIs y observabilidad. Adoptar microservicios sin esta madurez previa suele resultar en los problemas descritos como “monolito distribuido”, donde se obtiene la peor combinación de ambos mundos: alto acoplamiento y alta complejidad operacional [3].

Table 1: Síntesis comparativa entre Monolito Modular y Microservicios

Criterio	Monolito Modular	Microservicios
Escalabilidad	Limitada, generalmente uniforme	Granular, por servicio
Mantenibilidad	Buena si hay disciplina modular	Alta, con sobrecarga de coordinación
Complejidad operacional	Baja	Alta
Costos iniciales	Bajos	Altos
Curva de adopción	Baja	Alta
Ideal para	Equipos pequeños, dominios en evolución	Equipos grandes, escalado diferenciado

3.6 Síntesis comparativa

4 Aplicación al proyecto integrador (SGA educativo)

El proyecto integrador del equipo consiste en el desarrollo de un **Sistema de Gestión Académica (SGA)** orientado a instituciones educativas, el cual contempla módulos como gestión de matrículas, registro de calificaciones, administración de usuarios (estudiantes, docentes y administrativos), generación de reportes académicos y comunicación entre actores institucionales (notificaciones).

4.1 Características del dominio del SGA

El SGA educativo presenta varias particularidades relevantes para la decisión arquitectónica:

- **Carga variable por módulo:** durante periodos de matrícula o publicación de calificaciones, ciertos módulos (matrículas, notificaciones) experimentan picos de demanda significativamente mayores que módulos como la gestión de perfiles de usuario.
- **Equipo de desarrollo reducido:** al tratarse de un proyecto académico integrador, el equipo cuenta con recursos humanos y de infraestructura limitados.
- **Dominio en evolución:** los requisitos del sistema (nuevos reportes, integraciones con plataformas externas, módulos adicionales) probablemente cambiarán durante el desarrollo del proyecto, lo cual favorece arquitecturas que permitan refactorización ágil.
- **Necesidad de consistencia de datos:** procesos como la matrícula y el registro de calificaciones requieren consistencia transaccional fuerte (por ejemplo, no permitir matrículas duplicadas o calificaciones sin respaldo de matrícula previa).

4.2 Conexión con los conceptos investigados

Considerando el análisis desarrollado en la Sección 3, el SGA educativo del equipo se alinea de manera más natural con un **Monolito Modular** como punto de partida arquitectónico, por las siguientes razones técnicas:

En primer lugar, la **complejidad operacional** de los microservicios (Sección 3.3) representa un riesgo desproporcionado frente al tamaño del equipo y los plazos del proyecto integrador. Implementar orquestación de contenedores, *service discovery* y monitoreo distribuido para un sistema académico de alcance institucional limitado introduciría una sobrecarga que consumiría tiempo de desarrollo valioso sin un beneficio proporcional.

En segundo lugar, la **necesidad de consistencia transaccional fuerte** entre matrículas y calificaciones (Sección 4.1) se resuelve de manera natural en un monolito modular mediante transacciones de base de datos locales (ACID), mientras que en microservicios requeriría implementar patrones de consistencia eventual como *Saga* o *Outbox*, añadiendo complejidad significativa al dominio.

En tercer lugar, la **curva de adopción** (Sección 3.5) favorece directamente al equipo: un monolito modular permite que los integrantes se concentren en modelar correctamente los dominios de matrículas, calificaciones, usuarios y reportes como módulos cohesivos, aplicando los principios de bajo acoplamiento descritos en la Sección 2.1, sin la carga cognitiva adicional de la comunicación distribuida.

Sin embargo, el análisis de **escalabilidad granular** (Sección 3.1) sugiere una oportunidad concreta: el módulo de **notificaciones** –responsable de enviar correos y alertas durante periodos de matrícula y publicación de calificaciones– presenta un patrón de carga claramente diferenciado del resto del sistema. Por ello, se propone una arquitectura **híbrida**: un monolito modular como núcleo del sistema (matrículas, calificaciones, usuarios, reportes), extrayendo el módulo de notificaciones como un servicio independiente que se comunique mediante mensajería asíncrona (cola de eventos). Esta decisión aplica el principio de extracción incremental descrito en la Sección 2.3 [3], obteniendo el beneficio de escalado granular únicamente donde existe una necesidad real, sin asumir la complejidad operacional total de una arquitectura completamente distribuida.

4.3 Propuesta arquitectónica resumida

La arquitectura propuesta para el SGA del equipo se estructura de la siguiente manera:

- **Núcleo monolítico modular:** contiene los módulos de *Matrículas*, *Calificaciones*, *Usuarios* y *Reportes*, organizados con fronteras internas claras (paquetes independientes, interfaces bien definidas) y una base de datos relacional compartida con esquemas separados por módulo.
- **Servicio de notificaciones:** desacoplado del núcleo, expuesto mediante una API y comunicado a través de una cola de mensajes (por ejemplo, RabbitMQ o un broker equivalente), permitiendo su escalado independiente durante picos de demanda.
- **Estrategia de evolución:** si en el futuro otros módulos (por ejemplo, *Reportes*) demuestran patrones de carga o necesidades de escalado diferenciado, podrán extraerse siguiendo el mismo patrón aplicado a notificaciones, validando así la estrategia de migración incremental descrita en el marco teórico.

5 Conclusiones

A partir del análisis desarrollado, se concluye que la elección entre Monolito Modular y Microservicios no constituye una decisión binaria ni universal, sino que depende de factores contextuales

como el tamaño del equipo, la madurez operacional, los patrones de carga del sistema y la etapa de evolución del proyecto.

El Monolito Modular ofrece ventajas claras para equipos pequeños y dominios en evolución, al reducir la complejidad operacional y los costos iniciales, manteniendo al mismo tiempo una organización interna disciplinada que facilita la mantenibilidad. Por su parte, los Microservicios resultan idóneos cuando existe una necesidad demostrada de escalado granular y equipos suficientemente grandes para asumir la complejidad distribuida asociada.

Para el caso específico del SGA educativo del equipo, se determinó que un enfoque de **Monolito Modular con extracción selectiva de servicios** –comenzando por el módulo de notificaciones– representa la opción más equilibrada, al combinar la simplicidad operacional necesaria para un proyecto académico con la flexibilidad de escalar de forma independiente los componentes con mayor variabilidad de carga.

Finalmente, este análisis reafirma una idea central de la literatura consultada: la arquitectura de software debe evolucionar de manera incremental y fundamentada en evidencia real de las necesidades del sistema, evitando tanto la rigidez de un monolito mal estructurado como la complejidad prematura de una descomposición en microservicios sin justificación técnica suficiente.

References

- [1] Y. Abgaz, A. McCarren, P. Elger, D. Solan, N. Lapuz, M. Bivol, G. Jackson, M. Yilmaz, J. Buckley, and P. Clarke, “Decomposition of monolith applications into microservices architectures: A systematic review,” *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4213–4242, 2023, doi: 10.1109/TSE.2023.3287297.
- [2] K. Barde, “Modular monoliths: Revolutionizing software architecture for efficient payment systems in fintech,” *International Journal of Computer Trends and Technology*, vol. 71, no. 10, pp. 20–27, 2023, doi: 10.14445/22312803/IJCTT-V71I10P103.
- [3] M. Fowler, “Microservices: a definition of this new architectural term,” *martinfowler.com*, 2015. [En línea]. Disponible en: <https://martinfowler.com/articles/microservices.html>
- [4] G. Blinowski, A. Ojdowska, and A. Przybyłek, “Monolithic vs. microservice architecture: A performance and scalability evaluation,” *IEEE Access*, vol. 10, pp. 20357–20374, 2022, doi: 10.1109/ACCESS.2022.3152803.
- [5] H. Rodrigues, A. Rito Silva, and A. Avritzer, “Performance comparison of monolith and microservice architectures: An analysis of the state of the art,” in *Software Architecture. ECSA 2023 Tracks, Workshops, and Doctoral Symposium*, Lecture Notes in Computer Science, vol. 14590, Cham, Switzerland: Springer, 2024, pp. 185–199, doi: 10.1007/978-3-031-66326-0_12.